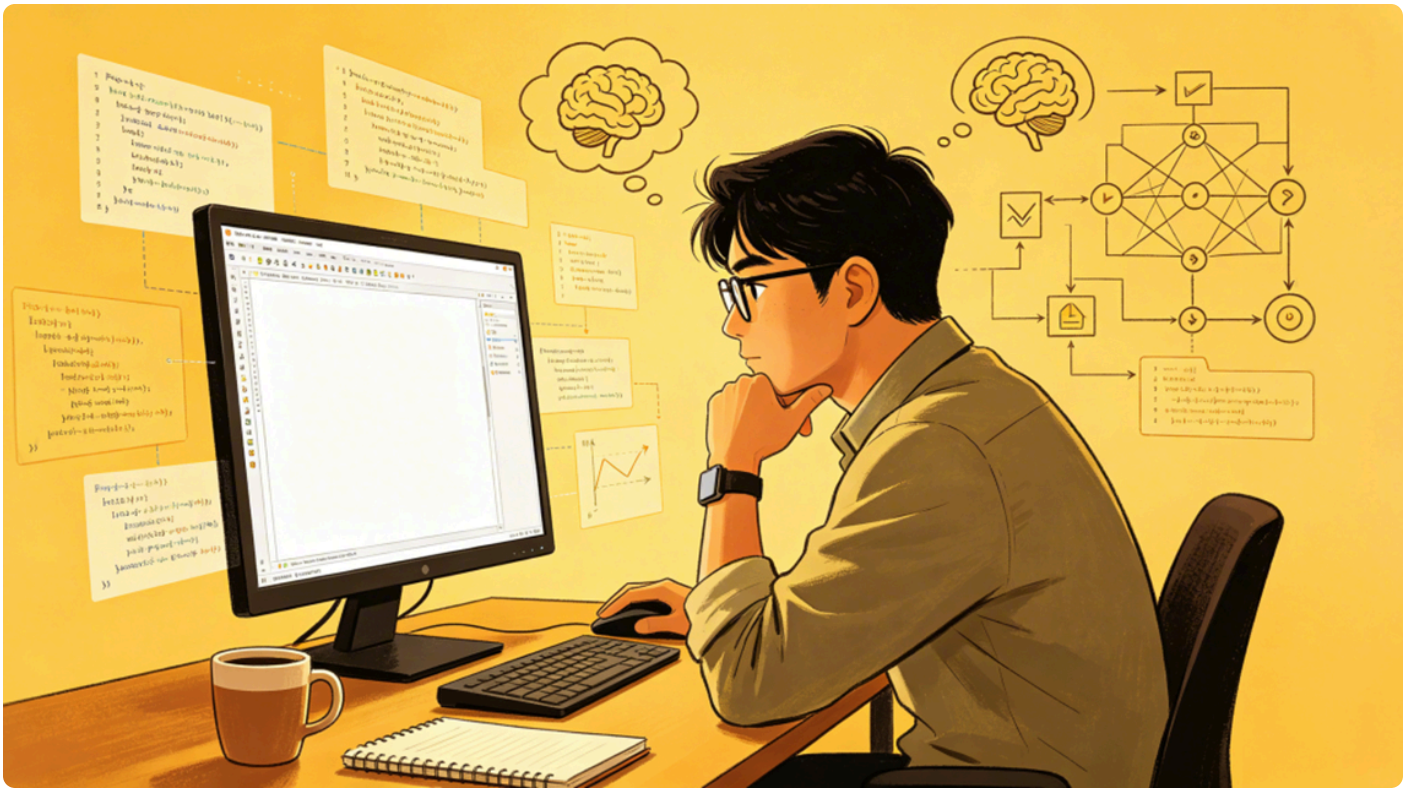
03｜产品定义：动第一行代码之前你该想清楚什么

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

前两讲我们建立了认知框架和方法论。从这一讲开始，正式动手做项目。但“动手”不是“写代码”。动手做的第一件事，是想清楚要做什么。

你可能觉得这不是已经定了吗？做一个简版 Dify，叫 Hify。但“做一个简版 Dify”这句话其实非常模糊。Dify 有几十个功能模块，你做哪些？不做哪些？目标用户是谁？部署在哪里？要扛多大的量？

这些问题不想清楚就开始写代码，会发生两件事：要么做着做着范围膨胀收不住，要么做到一半发现方向不对推倒重来。用 Claude Code 更危险，你说“帮我做个 Agent 管理”，它可能顺手把权限体系、版本控制、审计日志全给你加上，因为 Dify 有这些。你没定边界，它就替你定了。

所以我们这节课要做的事情就是：在动第一行代码之前，把产品边界、技术选型、运维预期全部想清楚，写进 CLAUDE.md。

注意，产品定义这一阶段就可以用 AI，不是只有写代码时才用。每一步我都会展示怎么让 Claude Code 协助完成。

先搞清楚业界标杆在做什么

做任何项目之前，第一步不是想“我要做什么”，而是看“业界最好的在做什么”。不是要抄，而是要理解这个领域的全貌。你连全貌都没有，怎么知道哪些功能是核心、哪些是边缘?

我们的标杆是 Dify，但 Dify 的功能很多，自己一个个去翻文档和产品页面，效率很低。这一步我直接让 Claude Code 帮忙：

> 帮我梳理 Dify（https://dify.ai）这个产品的核心功能模块，按类别分组，每个模块用一两句话说明它做什么。

Claude Code 很快给出了一份结构化的功能清单。

Dify 核心功能模块

模型管理

- **模型提供商** — 集成 OpenAI、Anthropic、Azure、本地模型等，统一管理 API Key 和调用配置。
- **模型切换** — 在应用内自由切换不同模型，支持 fallback 策略。

应用构建

- **聊天助手（Chatbot）** — 基于对话的单轮/多轮 AI 应用，可配置系统提示词和记忆窗口。
- **文本生成应用** — 单次输入→输出的批处理场景，适合写作、摘要、翻译。
- **Agent** — 赋予模型工具调用能力（搜索、代码执行、自定义 API），支持 ReAct 推理链。
- **工作流（Workflow）** — 可视化节点编排，将 LLM、工具、条件分支、循环组合成自动化流程。

知识库（RAG）

- **文档导入** — 支持 PDF、Markdown、网页等格式，自动切片、向量化存储。
- **检索策略** — 语义检索、全文检索、混合检索，可调相似度阈值和 top-k。
- **知识库关联** — 将知识库挂载到应用，让模型基于私有文档回答。

工具与插件

- **内置工具** — 网页搜索、代码解释器、DALL-E 图像生成等开箱即用。
- **自定义工具（API Tool）** — 通过 OpenAPI Schema 接入任意第三方 API，供 Agent 调用。

应用发布

- **WebApp** — 一键生成可分享的独立对话页面，无需前端开发。
- **API 发布** — 将应用暴露为标准 REST API，供外部系统集成。
- **嵌入组件** — 提供 iframe / JS 脚本，嵌入已有网站或产品。

运营与可观测

- **对话日志** — 记录所有用户会话，支持人工标注和纠错。
- **数据分析** — 活跃用户、消息量、Token 用量等核心指标统计。
- **标注与微调** — 对历史回答打标，用于改进提示词或训练数据。

协作与管理

- **工作空间** — 多成员协作，支持角色权限控制（Owner / Admin / Editor）。
- **环境变量** — 在应用内定义密钥、参数等变量，与代码解耦。

可以看到，这个答案看起来很完整。如果是了解 Dify 的人，那么其实就会知道这份是比较完整的。但是，如果我们不了解 DIfy 呢？此时原则是：**AI 给的信息不能直接信，你要验证**。比

如，我就拿着这个清单和 Dify 的官方文档、GitHub README 对了一遍，修正了几个过时的描述，补上了它遗漏的 MCP 协议支持。

最终梳理出的 Dify 功能全景：

模型管理：接入多个 LLM 提供商（OpenAI、Claude、Gemini、本地模型等），统一管理 API Key、模型参数、调用配额。

应用创建：用户可以创建不同类型的 AI 应用——聊天助手、Agent、文本生成、工作流。每个应用可以配置模型、提示词、工具。

工作流编排：可视化拖拽界面，把多个节点（LLM 调用、条件判断、代码执行、HTTP 请求）串成一个工作流。这是 Dify 的招牌功能。

RAG / 知识库：上传文档，自动切片、向量化、存储。对话时自动检索相关内容作为上下文。

工具集成：内置和自定义工具，Agent 可以在对话中调用。支持 API 工具和 MCP 协议。

对话与调试：完整的对话界面，支持流式响应、多轮对话。有调试模式可以看到推理过程。

发布与 API：每个应用可以发布为独立页面或 API，供外部系统调用。

运营功能：用量统计、日志追踪、成员管理、API Key 管理、多租户隔离。



Dify 八大功能全景图

看完这个清单，你就理解为什么“做一个简版 Dify”这句话很模糊了——随便拎出一块都能做几周。

从 Dify 到 Hify：功能取舍

功能全景有了，接下来是最关键的一步：做哪些、不做哪些。

我先让 Claude Code 做一轮初步分析：

> 我要基于 Dify 做一个简化版的 AI Agent 平台，叫 Hify。约束条件：一个人开发，面向团队内部 20–50 人使用，本地部署。请从刚才梳理的功能列表中，帮我判断哪些是必须做的核心功能，哪些可以砍掉，给出每个的理由。

Claude Code 给了一份很有结构的分析：

基于"一人开发 + 内部 50 人 + 本地部署"的约束，逐条判断：

必须做 ✅

模块	理由
模型提供商管理	基础设施，没有它什么都跑不起来
聊天助手（Chatbot）	团队最高频的使用场景，MVP 核心
知识库 + RAG	内部部署的核心价值所在——接入私有文档，这是团队用 Hify 而不是直接用 ChatGPT 的主要理由
API 发布	团队内其他系统集成的入口，开发者需要
对话日志	内部使用必须有审计能力，出了问题能追溯

可以做，但降级简化 ⚠️

模块	原版	简化方案	理由
工作流	可视化节点编排	先只支持线性流程（串行节点）	可视化拖拽开发量巨大，但简单流程够用
Agent	ReAct + 多工具调用	只支持挂载 1-2 个工具，不做复杂推理链	50 人团队的实际需求不会太复杂
自定义工具	OpenAPI Schema 接入	只支持 HTTP 工具，不解析 Schema	减少解析逻辑，手动填 URL + 参数即可
数据分析	完整指标看板	只做 Token 用量统计	控费是内部部署最实际的需求
权限管理	角色体系	只分管理员 / 普通用户两级	50 人不需要细粒度权限

砍掉 ❌

模块	理由
文本生成应用	聊天助手完全覆盖这个场景，一人开发不做重复入口
WebApp 发布	内部部署，直接用系统本身的界面即可，不需要独立分享页
嵌入组件	内部工具不存在"嵌入外部网站"的需求
标注与微调	需要训练基础设施，远超一人团队的维护能力
模型 Fallback	内部场景容错要求低，一个模型足够
工作空间多租户	只有一个团队，不需要隔离

你会发现 Claude Code 输出，不是简单的“做 / 不做”二分法，而是分成了三档：必须做、降级简化、砍掉。这个分档方式本身就有价值，我自己一开始想的是二分法。

它的分析里有一个判断让我重新思考了：

知识库 + RAG。我原本打算砍掉，整条链路太长，一个人做周期太长。但 Claude Code 指出：“接入私有文档，这是团队用 Hify 而不是直接用 ChatGPT 的主要理由。” 它说得对，如果只能通用对话，团队直接用 ChatGPT 就行了。私有知识的接入才是内部部署的核心价值。采纳，但降级：一期只支持 TXT 文档，分块用最简单的固定长度。

工作流也是类似的思路，不做可视化拖拽，只支持 JSON 配置的线性流程 + 条件分支，覆盖“问题分类→走不同路径”这种最常见场景。

当然也有我不同意的。它建议做两级权限（管理员 / 普通用户），但 50 人内部团队大家互相认识，在系统外约定就行，不值得为此每个功能都多一层权限逻辑。否决。

Claude Code 的分析帮我拓宽了思路，但最终取舍还是我基于自己的约束来做。这里我总结了一个方法论，叫做**“三问裁剪法”**。

第一问：没有它产品还成立吗？区分核心和边缘。模型管理、Agent 配置、对话引擎、工具集成、管理控制台，这五个砍掉任何一个产品都不成立。知识库和工作流砍掉产品能用但价值大打折扣，降级做。可视化拖拽、多租户、插件市场、计费，砍掉不影响核心链路。

第二问：做到什么程度够用？核心功能不做满。模型管理只做 CRUD + 连通性测试。Agent 配置只做选模型、绑工具、设提示词。对话引擎只做流式 + 多轮。知识库只支持 TXT + 固定分块。工作流只支持 JSON 配置。管理控制台能用就行不追求精美。

第三问：能不能一句话说清楚？Hify：一个可本地部署的 AI Agent 开发平台，支持多模型管理、Agent 配置、知识库 RAG、简版工作流、对话交互和 MCP 工具接入，面向团队内部小规模使用。

然后我就整理出了最终功能清单：

Hify 功能	决策	理由
多模型提供商管理	做	平台基础
Agent 创建与配置	做	核心载体
对话引擎（流式、多轮）	做	交互通道
MCP 工具接入	做	区别于聊天机器人的关键能力
管理控制台	做	用户操作入口
知识库 + RAG	降级做	内部部署的核心价值，一期只支持 TXT
工作流	降级做	JSON 配置，不做可视化拖拽
可视化工作流编排	降级做	前端拖拽实现极重
多租户 / 权限体系	不做	内部团队不需要隔离
插件市场 / 计费 / 微调	不做	超出一人开发范围

极客时间

技术选型：匹配定位，不追新

功能范围定了，接下来选技术栈。我同样让 Claude Code 参与：

> Hify 是一个 AI Agent 开发平台，一个人开发，本地部署，目标 20–50 人使用。帮我对比以下技术方案的优劣：1) Spring Boot + Vue 2) Go + React 3) Python FastAPI + React。重点考虑开发效率、生态成熟度、AI 领域 SDK 支持、运维复杂度。

Claude Code 给了一份四维度对比:



基于你的约束（一人开发、本地部署、AI 平台）逐维度对比：

四维对比

	Spring Boot + Vue	Go + React	Python FastAPI + React
开发效率	★★★ 生态完整但样板代码多	★★ 类型系统严格，写起来慢	★★★★★ 最快，AI 相关代码几乎都有现成库
生态成熟度	★★★★★ 最成熟，企业级方案齐全	★★★ 成熟但 AI 生态薄	★★★★ AI 生态第一，Web 生态次之
AI SDK 支持	★★ 有 Java SDK，但社区活跃度低	★ 基本靠手写 HTTP 调用	★★★★★ LangChain、LlamaIndex、Instructor 等原生支持
运维复杂度	★★★ JVM 吃内存，打包成 jar 尚可	★★★★★ 单二进制，资源占用最低	★★★★ 依赖管理略繁，但 Docker 一键搞定

各方案核心问题

Spring Boot + Vue

- 优势：你最熟（当前项目就是这个栈），MyBatis-Plus、Spring Security 等开箱即用
- 问题：做 AI 平台要集成 RAG、向量检索、流式输出，Java 生态里没有好用的一站式方案，很多东西要自己造轮子

Go + React

- 优势：性能好，部署极简（一个二进制）
- 问题：一人开发 AI 平台选 Go 是最难的路，AI 相关 SDK 几乎没有，LLM 调用全靠手写，得不偿失

Python FastAPI + React

- 优势：LangChain/LlamaIndex 直接用，向量库（Chroma、Qdrant）有成熟 Python 客户端，流式输出 SSE 写起来最简单
- 问题：前端还是要写 React，纯后端是 Python 但你可能不熟

结论

情况	推荐
你熟悉 Java，不想学新语言	**Spring Boot + Vue**，但 AI 能力层单独起一个 Python 微服务（只负责 RAG + 向量检索）
你愿意学或已会 Python	**Python FastAPI + React**，这是做 AI 平台最省力的选择
追求极简运维	**Go + React**，但要做好 AI 功能全靠自己手写的心理准备

一句话建议：你当前项目已经是 Spring Boot + Vue，不用推倒重来。把 AI 重逻辑（RAG、向量化、复杂 prompt 链）抽成一个 Python 小服务，Spring Boot 做业务编排和权限，这是一人开发最务实的拆法。

你会发现结论很明确：Python FastAPI 在 AI 生态上碾压其他两个——LangChain、LlamaIndex，向量库客户端都是 Python 原生的，Java 和 Go 在这方面差距很大。它推荐 FastAPI，还给了一个折中方案：Spring Boot 做业务编排，AI 重逻辑（RAG、向量化）抽成一个 Python 小服务。

这个折中方案思路不错，但我不同意。一个人维护两套技术栈（Java + Python），两个服务之间还要做通信、部署、调试，复杂度翻倍。一个人开发最忌讳的就是技术栈分裂。

我追问了一句：

> 一个人做企业级后端，Spring Boot 和 FastAPI 在工程化能力上差距有多大？考虑异常处理体系、事务管理这些方面。

它的分析很诚实：FastAPI 在这些方面需要自己搭建更多基础设施，Spring 的生态已经把这些都解决了。

我的判断：选 Spring Boot + Vue，不拆 Python 服务。理由有三个。

1. 我最熟这个栈，一个人开发效率第一。
2. AI SDK 用 Java 版本够用，OpenAI、Claude 都有 Java SDK，RAG 的向量化调 API 就行，不依赖 LangChain。
3. 一个人一套技术栈，不分裂。

AI 生态的劣势确实存在，但工程化能力和技术栈统一性的优势更大。技术选型不是选最好的技术，是选最匹配当前约束的技术。

最终技术栈：

后端：Spring Boot 3.x + MyBatis-Plus + MySQL 8.x + Redis 7.x。工程化成熟，一个人能快速交付完整系统。

前端：Vue 3 + TypeScript + Element Plus。管理后台场景成熟度最高。

容器化：Docker + Docker Compose。最稳、最简。

提前想清楚怎么跑

很多人做项目只想功能，不想运维。功能做完了往服务器上一扔，等出问题了才发现很多事没有提前想。

这一步我先自己想一遍，然后让 Claude Code 帮我查漏补缺：

> Hify 是一个 AI Agent 平台，Docker Compose 本地部署，目标 20-50 人同时在线，主要压力在对话接口（流式 SSE）。帮我估算 QPS、建议缓存策略、列出需要提前考虑的运维事项。

Claude Code 给的分析很实在。几个关键结论：

1. **QPS 极低，瓶颈不是并发**。50 人在线，活跃率 60%，每人每分钟 2 条消息，峰值也就 1 QPS，加上 RAG 检索等内部调用放大 3 倍，也就 3-5QPS。Spring Boot 单实例轻松应

对。真正的瓶颈是 LLM 响应延迟（3–30 秒 / 次）占用连接——这不是 QPS 问题，是长连接管理问题。

2. **SSE 长连接是唯一需要认真对待的压力点**。50 个并发 SSE 连接，每个持续 10–30 秒，意味着 50 个线程被长期占用。Spring Boot 默认 200 线程够用，但要注意：用 SseEmitter 不要用普通 ResponseBody 阻塞线程，设 60 秒超时防止僵尸连接，Nginx 反向代理缓冲（proxy_buffering off）否则流式数据会被攒批再发。这几个坑我自己没全想到，尤其是 Nginx 缓冲那个——不关的话用户看不到打字机效果，要等 LLM 全部输出完才一次性吐出来。
3. **缓存策略简单直接**。模型提供商配置和 Agent 配置变更频率极低，用 Redis 缓存。LLM 响应不要缓存——同一问题不同时间答案可能不同。对话历史直接读数据库，50 人规模数据量小，缓存收益不大。
4. **数据持久化是硬性要求**。MySQL 数据目录、向量数据库数据目录、上传的文档文件——必须挂载 volume，容器内不存储任何数据。这条看起来是常识，但真有人忘了挂载，容器一重启数据全没了。

这里有一个环节很重要，你得看得懂上面四个建议。如果你看不懂，那么其实整个项目就会失控。比如因为短期 QPS 不高，我放弃了做缓存，因为缓存会增加开发的复杂度。

最终的运维预期：

部署方式：Docker Compose 一键启动。JVM 内存设上限（–Xmx512m），防止容器被 OOM Kill。

用户量：20–50 人同时在线，单机部署够用。

QPS：峰值 3–5QPS，瓶颈在 LLM 长连接不在并发。

监控：起步用 Spring Boot Actuator + 日志，后期接 Prometheus + Grafana。

扩容：一期单机。架构上不堵死扩容的路——第 04 讲会做模块化设计，为后续拆分留口子。

这些判断不需要很精确，但它们会反向影响技术选型和架构决策。比如“瓶颈在长连接不在并发”这个判断，直接决定了第 04 讲 LLM 调用要用独立线程池做隔离。“一期单机”这个判

断，决定了不需要微服务。所以，运维预期不是项目最后才想的事，它从一开始就在影响你的每一个设计决策。

写进 CLAUDE.md

前面想清楚的所有东西，产品定义、功能范围、技术选型、运维预期，都要落成文字，写进 CLAUDE.md。

这一步也让 Claude Code 帮忙：

> 根据我们的讨论，帮我把 Hify 的项目概述写进 CLAUDE.md 的“项目概述”部分。包括产品定位、做什么、不做什么、技术栈、部署与运维预期。格式简洁明了。

它生成了一版初稿，我检查修改后定稿：

复制代码

```
## 项目概述

Hify 是一个简版的 AI Agent 开发平台（参考 Dify），可本地部署，
面向团队内部小规模使用（20-50 人同时在线）。

### 做什么
- 多模型提供商管理（OpenAI、Claude、Gemini、Ollama）
- Agent 创建与配置（选模型、绑工具、设系统提示词）
- 对话引擎（流式响应、多轮对话、上下文管理）
- 知识库 + RAG（一期只支持 TXT 文档，固定长度分块）
- 简版工作流（JSON 配置，线性 + 条件分支，不做可视化拖拽）
- MCP 工具接入（Agent 可通过 MCP 协议调用外部工具）
- 管理控制台（模型管理、Agent 配置、对话界面）

### 不做什么
- 不做可视化工作流拖拽编排
- 不做多租户 / 权限体系
- 不做插件市场、计费系统
- 不做文本生成应用、WebApp 发布、嵌入组件
- 不做标注与微调

### 技术栈
后端：Spring Boot 3.x + MyBatis-Plus + MySQL 8.x + Redis 7.x
前端：Vue 3 + TypeScript + Vite + Element Plus
容器化：Docker + Docker Compose
```

```
### 部署与运维预期
- Docker Compose 本地一键部署，JVM 内存设上限（-Xmx512m）
- 目标：20-50 人同时在线，峰值 3-5 QPS，瓶颈在 LLM 长连接
- 缓存：Redis Cache-Aside（配置信息 + 会话上下文）
- 监控：起步 Actuator + 日志，后期 Prometheus + Grafana

```

CLAUDE.md 不是一次性写好就不动了。随着项目推进，你对产品的理解会加深，可能会调整功能范围、改变技术决策。每次有变化，回来更新。这和第 02 讲的 SDD 闭环是一样的道理，规范是活的。

总结

这节课我们做了一件事：在动第一行代码之前，把产品的边界想清楚了。

你会发现，这其实是一个问对问题、鉴别问题的过程。其实最难的是这一步，怎么问对问题，怎么鉴别问题。这都是需要积累的。这一点，这门课不能帮到你太多，只能尽量告诉你我是怎么想的。而在举一反三的过程，需要你不断的学习、积累和反思。

整个过程五步：调研标杆（让 Claude Code 梳理 Dify 功能全景）→ 功能取舍（三问裁剪法，Claude Code 做初筛你拍板）→ 技术选型（Claude Code 做方案对比，你基于约束选择）→ 运维预期（Claude Code 帮你查漏补缺）→ 落成文字（写进 CLAUDE.md）。

几个小时就能完成，但能帮你在后面几周的开发里避开大量弯路。而且你注意到了，整个过程中 Claude Code 都在协助你，但没有一步是它替你做决策的。它梳理了 Dify 的功能清单，你来验证修正。它建议保留 RAG，你否决了。它推荐 FastAPI，你选了 Spring Boot。每一步的最终判断都是你做的。

你做任何项目，都可以按这五步走一遍。Hify 的具体定义是我们这个项目的，你照搬没用，但这五步的拆解思路是通用的。

下一讲，我们进入架构设计——模块怎么分、数据怎么流转、外部调用怎么处理。

思考题

如果你要做的不是 Agent 平台，而是你自己工作中真正需要的一个系统，试着用今天的五步走一遍。特别是第二步的三问裁剪法，花 10 分钟列出“做什么”和“不做什么”的清单。你会发现很多你以为“肯定要做”的功能，用第一个问题一过滤就可以砍掉。如果你在某一步用了 Claude Code 辅助，把你的指令和它的输出也分享出来，我们一起看看它给的建议靠不靠谱。

期待你的分享。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (24)

最新　精选



黄马
2026-04-02 来自中国香港

没有经验这活还真干不了

> 作者回复: 是的，没问题。所以我们课程的假设是有一定的编程和项目经验的同学。然后如何的CC的辅助下，能够加速自己的成长。
> 以前我们的的思路是，找有经验的教我们，找老员工，找leader。比如我这代人，刚毕业那会，就是问问问，翻书，翻百度。架构设计基本靠经验积累。
> 但是现在我们想达到的目的是，假设是有cc 这样一个有经验的、有耐心的导师的情况下，如何在cc的辅助下快速拿到结果或者说成长。
> 让它辅助我们去拆解、成长。包括我自己，在不熟悉一个领域时，如何快速了解同样的道理。

13



巴马
2026-04-14 来自四川

老师，现在有SDD规范驱动开发的框架，您的这套编程思想会用到SDD规范驱动开发的现成框架吗？比如spec-kit 、OpenSpec等

作者回复: 这些框架我了解，也研究过。我没有用它们，主要有下面几个原因：
第一，它们太通用，我的规范需要足够具体。Spec Kit 可以生成规范骨架，但骨架是通用的。Hify 这个项目，哪些模块用哪种分层、接口返回什么格式、异常怎么处理、索引怎么建，这些规范是我自己一条一条打磨出来的，和项目深度绑定，套模板反而要做大量裁剪。
第二，工具本身还很早期，带来的成本不低。Spec Kit 现在还在 0.3 版本，绑定 AI 工具之后不好切换，文档跟不上迭代速度，学习成本是真实的。对一个独立开发的项目来说，这个成本不合算。
第三，CLAUDE.md 就是我的 SDD 框架。它定义了架构规范、代码组织、命名约定、禁止事项、性能要求，Claude Code 每次启动自动加载，这就是规范驱动开发的完整闭环。不需要外部框架，因为我已经有了相当于适合自己需求的实现。
我个人的判断是：工具是手段，不是目的。 如果你的 CLAUDE.md 已经能让 AI 稳定在你设计的框架里工作，效率足够高，没有必要为了用框架而用框架。但如果你是团队协作、规范需要跨项目复用，或者你刚开始做 SDD 不知道怎么组织规范文件，Spec Kit 这类工具就是有价值的。

8

sid
2026-04-02

老师，想请教一下：在AI辅助开发的前提下，后端靠 AI 补前端很容易，前端补后端反而难。想问问在 AI 开发时代，前端想转全栈，最该补哪些后端知识呢?

作者回复: 如果你让我列，我肯定可以列一对你熟悉的。也就是常说的，后端语言，数据库，redis，这些。
但是单纯讲出来有点空了，是吧。
所以，我觉得还是那句老话，从项目出发去学是效果最好的。项目需要什么，就补什么。触类旁通，旁征博引的思维去学习。
比如你可以试着跟着我们这个项目，开始学后端。哈哈哈。我个人觉得挺好的。先学后端思维，然后学db、mq、cache等等。一步一步来。

7

ikimiy
2026-04-18 来自上海

帮我梳理 Dify（https://dify.ai）这个产品的核心功能模块，按类别分组，每个模块用一两句话说明它做什么。 老师 你做的hify是以dify为参照来整理出功能模块后一步步走下去的，但是如果我需要做的系统并没有参照物，那应该如何开始呢，这个功能列表需要自己去写出来，然后才能走什么能做，什么不能做这个步骤是吧

作者回复: 没有参考，也可以直接问。因为理论上我们现在做的任何事情，cc 都可以给建议。你把它当作一个专家的角度来问。可能不是你这个方向的专家。这种情况就需要你有更详细的判断力了。

3

巨熊
2026-06-17 来自广东

老师。请教一下。我的后端技术栈主要是 nodejs。对 JAVA 基本不熟。项目中如果采用 nest js 来代替后端开发框架。您认为会存在卡点吗?

作者回复: 我理解不会有卡点，因为你会nodejs，即使对java不熟，花点时间学习也会熟悉的，不是啥卡点。
唯一的卡点就是：你不想花时间。哈哈哈

共 2 条评论>

2

Geek_9d61fc
2026-05-05 来自上海

老师，做什么和不做什么为何是写进CLAUDE.md, 而非spec.md？ CLAUDE.md应该是总体规范?

作者回复: 我认为CLAUDE.md和spec.md都是SDD的一种表现形式。
我写这门课的时候，是想从CLAUDE.md开始，渐渐的把SDD融入到项目中讲解，然后内化为习惯。
所以在一开始，就会把一些东西都放到CLAUDE.md中。到后面就会慢慢拆出来。

1

菲尔德
2026-04-20 来自贵州

技术栈是不是少了向量化库

作者回复: 这个有的，向量数据库用的pgvector

共 2 条评论>

1

AnMin
2026-06-29 来自美国

感觉非常适合等1-2门语言的AI产品经理，直接出MVP，快速验证；

作者回复: 我写课的时候，也觉得蛮适合的，很适合快速做出一个作品，自己不费什么力。






奔跑的牛牛

2026-06-25 来自辽宁

课程的项目对移动端开发的不太友好， 没做过后端， 一直是Android Framework车载方向开发，后端技术栈了解太少， 感觉融入不了

作者回复: 这点确实是，我的背景就是后端，这门课是从后端的角度来设计的。确实不太适合移动端背景的同学






grace

2026-06-20 来自上海

这节课是不是对于零基础，编程弱的同学不适合？？？

作者回复: 这就是得看一下对“零基础，编程弱”的定义了。如果一点基础都没有，确实会会看的压力有点大。我是按照有点基础的标准写的～。不过我建议可以试试，边看边学效果应该还行。